

Vocably

Cómo se construyó una aplicación de vocabulario en cuatro días, y todo lo que aprendí por el camino

Diario técnico y de aprendizaje

Yijun Wang · septiembre de 2026

112 commits · 440 pruebas · 181.103 entradas de diccionario

De qué va este documento

Este documento cuenta, de principio a fin, cómo se hizo **Vocably**: una aplicación web personal que extrae vocabulario en inglés de un PDF con la API de Claude y lo devuelve para repasarlo con repetición espaciada. No es un manual de uso —eso está en el README del repositorio— sino el relato de las decisiones, los errores, los conceptos de ingeniería que hubo que entender y, sobre todo, el **catálogo de herramientas nuevas** que aparecieron por el camino: piezas de software, servicios y métodos de trabajo que antes de este proyecto no estaban en la caja.

Todo lo que aquí se afirma está comprobado contra el repositorio real: el historial de git, las especificaciones, las migraciones, el código y la salida de la suite de pruebas.

CONTENIDO

Índice

1	Resumen en una página
2	Qué es Vocably y qué problema resuelve
3	El proyecto en cifras
4	Cronología: cuatro días, cuatro entregas
5	El método de trabajo: especificar, planificar, construir, revisar
6	La arquitectura, y por qué es así
7	Fase 1 — Extraer vocabulario de un PDF
8	Fase 2 — Repasar con repetición espaciada (FSRS)
9	Fase 3 — El diccionario propio: 181.103 entradas por 0 €
10	Fase 4 — Elegir qué y cuánto repasar
11	Catálogo de herramientas descubiertas
12	Conceptos de ingeniería que hubo que entender
13	Errores cometidos y lo que enseñaron
14	Lo que cuesta y lo que es gratis
15	Estado actual, límites conocidos y qué viene después
16	Glosario
A	Apéndice: mapa de ficheros
B	Apéndice: comandos útiles

CÓMO LEER ESTE DOCUMENTO

Las secciones 1 a 4 cuentan el **qué** y el **cuándo**. La 5 cuenta **cómo se trabajó**, que es probablemente lo más transferible a cualquier otro proyecto. Las secciones 6 a 10 son el recorrido técnico fase por fase. La **sección 11 es el catálogo de herramientas**: si solo se lee una parte, que sea esa. Las secciones 12 y 13 recogen los conceptos y los tropiezos.

Los recuadros verdes marcados **LO QUE APRENDÍ** destilan cada apartado en una idea que se puede llevar a otro proyecto.

SECCIÓN 1

Resumen en una página

Vocably nació para automatizar algo que ya se hacía a mano: copiar un prompt en un chat, pegarle unas páginas de un libro en inglés y pedirle el vocabulario de un nivel concreto. Eso funcionaba, pero el resultado se quedaba en una conversación y no había forma de repasarlo. La aplicación convierte ese flujo manual en un producto: se sube el PDF, se eligen páginas y nivel, y el vocabulario queda guardado en una base de datos y programado para volver a aparecer justo antes de que se olvide.

Se construyó entre el **6 y el 9 de septiembre de 2026** —cuatro días naturales— en 112 commits, con una aplicación Next.js desplegada en Vercel y Postgres en Neon: 5.671 líneas de código de aplicación cubiertas por 5.534 líneas de pruebas. La suite completa son **440 pruebas en 42 ficheros**, y ninguna llama a la API de Claude, ni a un traductor externo, ni a una base de datos real: todo corre con respuestas grabadas y un Postgres en memoria.

El proyecto salió en cuatro entregas, cada una con su especificación escrita antes de tocar código:

- 1 Extracción.** Recortar un rango de páginas del PDF *en el navegador*, mandarlo por lotes a claude-opus-5 con salida estructurada, y guardar los términos deduplicados en Postgres, con el coste real a la vista.
- 2 Repaso.** Una pantalla de tarjetas gobernada por el algoritmo FSRS de repetición espaciada, instalable en el móvil como una app, y que *no gasta ni un céntimo* de API.
- 3 Diccionario.** Una segunda puerta de entrada al vocabulario: 181.103 entradas de Wikcionario cargadas en la propia base de datos, con un traductor gratuito de refuerzo y Claude solo como botón opcional.
- 4 Colecciones.** Una pantalla previa al repaso para elegir de qué grupo salen las palabras y cuántas entran en la sesión.

Lo que sigue cuenta cómo se llegó a cada una de esas cuatro cosas, qué se rompió por el camino y qué herramientas hubo que aprender para llegar hasta aquí.

4 días de trabajo	112 commits	440 pruebas	24 herramientas nuevas	0 € coste fijo al mes
---	-----------------------------------	-----------------------------------	--	---

SECCIÓN 2

Qué es Vocably y qué problema resuelve

El problema de partida

Aprender vocabulario de un idioma leyendo tiene un cuello de botella que no está en la lectura, sino en lo que viene después. Uno lee un artículo o un capítulo, subraya seis expresiones útiles y, dos semanas más tarde, no recuerda ninguna. El material existe; lo que falta es un sistema que lo capture sin fricción y lo devuelva en el momento justo.

Había además un problema más concreto: los verbos frasales y las expresiones idiomáticas no se pueden extraer con un diccionario ni con una expresión regular. *Come across* no significa «venir a través»; hace falta algo que entienda el contexto de la frase. Ese es el único sitio del proyecto donde un modelo de lenguaje resulta imprescindible, y por eso es el único sitio donde se usa de forma obligatoria.

La solución en cuatro movimientos

Paso	Qué ocurre	Dónde
1. Capturar	Se sube un PDF, se elige un rango de páginas y un nivel del MCER (A1 a C2). Claude devuelve palabras, verbos frasales y expresiones con traducción, el contexto original y un ejemplo de uso.	/extraer
2. Buscar	Alternativa sin PDF y sin coste: se busca una palabra suelta en un diccionario propio de 181.103 entradas y se añade con un botón.	/diccionario
3. Guardar	Los términos viven en Postgres, deduplicados. Se pueden corregir a mano sin perder el progreso de repaso de esa palabra.	/biblioteca
4. Repasar	Tarjetas de una en una, con cuatro valoraciones, gobernadas por FSRS. Instalable en el móvil.	/repaso

Las restricciones que dieron forma a todo

Tres decisiones tomadas el primer día condicionaron el resto del proyecto, y conviene entenderlas porque explican casi todas las rarezas del código:

- **Un solo usuario.** Sin registro, sin cuentas, sin roles: se entra con una única contraseña. Eso elimina de golpe toda una capa de complejidad —tablas de usuarios, permisos, recuperación de contraseña— a cambio de aceptar limitaciones conscientes que están documentadas.
- **Coste casi cero.** Vercel gratuito, Postgres gratuito en Neon, diccionario gratuito de Wikcionario, traductor gratuito. Lo único que se paga es la API de Claude, y solo en la extracción.
- **El PDF nunca se almacena.** Se recorta en el navegador, se manda el trozo, se extrae y se descarta. No hay gestión de ficheros, ni almacenamiento, ni una política de borrado que escribir.

LO QUE APRENDÍ

Las restricciones no son el enemigo del diseño: son el diseño. «Un solo usuario» y «el PDF no se guarda» eliminaron más trabajo del que añadió cualquier funcionalidad. Antes de empezar a construir, la pregunta más rentable no es «qué debería hacer» sino «qué puedo prometer que *no* hará».

SECCIÓN 3

El proyecto en cifras

Todos estos números salen de medir el repositorio, no de estimarlos.

Tamaño y forma del código

Medida	Valor	Comentario
Commits	112	Del 6 al 9 de septiembre de 2026
Ficheros añadidos	148	34.003 líneas insertadas desde el primer commit
Código de aplicación	5.671 líneas	TypeScript y TSX en app/, components/, lib/, db/ y scripts/
Código de pruebas	5.534 líneas	Prácticamente uno a uno con el código de aplicación
Pruebas	440 en 42 ficheros	Todas pasan; la suite tarda 43 segundos
Rutas de API	10	Bajo app/api/
Pantallas	6	Entrada, extracción, biblioteca, repaso, diccionario y raíz
Tablas en la base	7	Fuentes, términos, apariciones, estado de tarjeta, histórico, ajustes y diccionario
Migraciones	6	De 0000_fluffy_pixie a 0005_organic_scorpion

Documentación escrita

Un dato que sorprende: **se escribieron más palabras de documentación que de código**. Las cuatro especificaciones y los cuatro planes de implementación suman unas 45.900 palabras, y casi todas se escribieron *antes* de la línea de código correspondiente.

Documento	Palabras	Papel
Especificación general	1.678	Qué es la app, decisiones, arquitectura, modelo de datos
Especificación de la fase 2 (repaso)	1.276	El repaso con FSRS y los ajustes
Especificación del diccionario	2.630	Las cuatro fuentes, la caché, la pista en inglés
Especificación de colecciones	2.226	La pantalla previa y el tamaño de sesión
Los cuatro planes de implementación	34.086	Tarea por tarea, con ficheros, interfaces y pruebas
README	3.999	Manual de uso, despliegue y limitaciones conocidas

LO QUE APRENDÍ

La proporción documentación/código no es un lujo cuando quien escribe el código es un agente. Un plan detallado —con los ficheros que toca cada tarea, las firmas que produce y las pruebas que debe pasar— es lo que evita que doce tareas independientes acaben contradiciéndose. El coste de escribirlo se recupera la primera vez que dos tareas se pisan y el plan ya había detectado el choque.

Ritmo de trabajo

Día	Commits	Qué salió ese día
6 de septiembre	25	Diseño general, plan de la fase 1, base del proyecto, extracción y biblioteca
7 de septiembre	49	Fase 2 completa (FSRS, repaso, PWA, ajustes) y arranque del diccionario
8 de septiembre	37	Diccionario terminado y fundido en main; colecciones de repaso
9 de septiembre	1	Ajuste del tiempo de lectura de la frase de entrada

SECCIÓN 4

Cronología: cuatro días, cuatro entregas

El proyecto no se construyó de un tirón. Cada entrega repitió el mismo ciclo —hablarlo, especificarlo, planificarlo, construirlo, revisarlo, fundirlo— y cada una empezó cuando la anterior ya estaba en uso. Eso está escrito literalmente en la primera especificación: «el plan de implementación que sigue a este documento cubre solo la fase 1; las fases 2 y 3 tendrán el suyo propio, escrito cuando la anterior esté en uso».

Día 1 — 6 de septiembre: del papel a la primera extracción

El día empezó sin una sola línea de código: el primer commit del repositorio se llama «Añadir diseño de AppVocabulario» y es un documento de mil seiscientas palabras. El segundo es el plan de la fase 1. Solo entonces aparece «feat: base del proyecto Next.js y normalización de términos».

En ese mismo día entraron el esquema de la base de datos, el guardado con deduplicación, la llamada a Claude con salida estructurada y el cálculo del coste real, el recorte del PDF en el navegador, la pantalla de extracción con lotes en secuencia, la entrada con contraseña única y la biblioteca editable. Terminó con una medición: «Sustituir la estimación de coste por una medición real».

Día 2 — 7 de septiembre: el repaso, y la app en el móvil

El día más denso: 49 commits. Diseño y plan de la fase 2, el puente con `ts-fsrs`, la cola del día, las rutas de cola y respuesta, la orquestación de la sesión en el navegador, la pantalla de repaso, el tope de tarjetas nuevas, el manifiesto para instalar la app en el móvil, un rediseño completo de la extracción y de la biblioteca, y el cambio de nombre del proyecto a **Vocably**. Al final del día ya se podía repasar vocabulario desde el teléfono.

Ese mismo día arrancó la fase 3 con dos commits de documentación: la especificación del diccionario y su plan en doce tareas.

Día 3 — 8 de septiembre: el diccionario y las colecciones

Se construyó el diccionario entero: convertir una línea del volcado de Wikcionario en filas, la tabla, el cargador, la búsqueda con variantes del lema, el traductor gratuito con caché, el botón opcional de afinar con Claude, la pantalla y la posibilidad de guardar dos acepciones del mismo término. Se fundió en `main` y, sin pausa, empezó la cuarta entrega: elegir qué y cuánto repasar antes de empezar.

Día 4 — 9 de septiembre: pulir

Un único commit, y muy revelador del tipo de detalle que ocupa el final de un proyecto: «Dar tiempo a leer la frase de la entrada».

EL PATRÓN QUE SE REPITE CUATRO VECES

Especificación (qué y por qué) → plan por tareas (cómo, con pruebas) → construcción tarea a tarea → revisión de código → corrección de hallazgos → documentación actualizada → fusión. Ninguna de las cuatro entregas se saltó un paso, y las cuatro terminaron con un commit de tipo «docs:» corrigiendo lo que la especificación había prometido y la realidad cambió.

SECCIÓN 5

El método de trabajo: especificar, planificar, construir, revisar

Esta es, con diferencia, la parte más transferible del proyecto. El código de Vocably sirve para Vocably; el método sirve para lo siguiente que se construya.

Paso 0. Conversar antes de escribir

Antes de la primera especificación hubo una conversación estructurada cuyo único objetivo era sacar a la luz decisiones que de otro modo se toman por inercia: ¿solo extraer, o también repasar? ¿web o móvil? ¿un usuario o varios? ¿qué algoritmo de repetición espaciada? Cada respuesta quedó escrita en una tabla de la especificación con su motivo al lado:

Decisión	Elección	Motivo
Alcance	Extraer y repasar	El objetivo es adquirir vocabulario, no producir tablas
Plataforma	Web desplegada, con móvil	El repaso ocurre en huecos muertos, fuera del escritorio
Extracción	API de Claude	Es lo único que entiende contexto: verbos frasales y expresiones
Usuarios	Solo el propietario	Menos código, antes en las manos del usuario
Repaso	Repetición espaciada (FSRS)	El método con más respaldo para memoria a largo plazo
Despliegue	Vercel, plan gratuito	Cero mantenimiento; solo se paga la API

El valor de esa tabla no es la decisión: es **el motivo**. Tres días después, cuando alguien propone «¿y si la traducción la hace también la IA?», la tabla contesta sola.

Paso 1. La especificación: qué, y sobre todo por qué

Cada entrega tiene un documento de diseño con la misma estructura: qué es, decisiones tomadas con su motivo, las pantallas, el modelo de datos, los límites de la plataforma y lo que explícitamente *no* se va a construir. Son documentos vivos: los cuatro llevan una línea de «Estado» y tres de ellos llevan además **enmiendas fechadas**, marcadas dentro del texto con (*enmienda*), que registran las decisiones que cambiaron durante la construcción.

LA ENMIENDA, UNA IDEA PEQUEÑA Y MUY ÚTIL

Cuando construir algo demuestra que la especificación estaba equivocada, hay dos malas salidas: reescribir el documento como si nunca se hubiera equivocado, o dejarlo mintiendo. La tercera es anotar la enmienda en su sitio, con fecha y motivo. Así el documento sigue siendo verdad y además conserva el aprendizaje. En este proyecto hay un commit dedicado solo a esto: «docs: enmendar la especificación del diccionario con lo que cambió al construirlo».

Paso 2. El plan: el proyecto partido en tareas verificables

De la especificación sale un plan de implementación —entre siete y nueve mil palabras— que parte el trabajo en tareas numeradas. Cada tarea declara cuatro cosas:

- **Ficheros:** qué crea, qué modifica y qué fichero de pruebas le corresponde.
- **Interfaces:** qué consume de otras tareas y qué produce para ellas, con la firma exacta. Por ejemplo: «Produce: `normalizeTerm(term: string): string` — la clave de deduplicación que usan las tareas 3 y 7».
- **Pasos:** instrucciones concretas, a veces con el comando literal que hay que ejecutar.
- **Pruebas:** cuántas y cuáles, escritas *antes* que el código que las hace pasar.

El plan también fija **restricciones globales** que ninguna tarea puede violar: el modelo es `claude-opus-5` y no se sustituye sin decisión explícita; el lote es de 5 páginas; el PDF original nunca se guarda; los secretos no llevan nunca el prefijo `NEXT_PUBLIC_`; las pruebas no llaman a la API; la interfaz está en español; los niveles permitidos son exactamente A1 a C2.

Paso 3. El escaneo previo de conflictos

Antes de construir nada, el plan se audita contra sí mismo. Queda registrado en un fichero de seguimiento que cruza cada par de tareas que comparten un fichero o una interfaz y comprueba que encajan. Un extracto real de la fase 1:

Tareas	Produce -> consume	Resultado	
1 -> 3, 7	<code>normalizeTerm(string): string</code>	OK, firma coincide	
3 -> 5	<code>tipo ExtractedTerm</code>	OK, exportado	
5 -> 7	<code>{items, inputTokens, outputTokens, costUsd}</code>	OK, mismos nombres	
9 -> 7	<code>cuerpo {pdfBase64,title,pageStart,...}</code>	OK, encaja con Body	

Y una segunda comprobación, aún más simple y que encontró un defecto de verdad: contar si el número de pruebas que declara cada tarea coincide con las que realmente enumera. La tarea 4 decía nueve y tenía once. Hay un commit solo para eso: «chore: corregir el número de pruebas esperado en la tarea 4».

LO QUE APRENDÍ

Un plan es un artefacto que se puede *verificar*, no solo leer. Cruzar productores con consumidores y contar pruebas declaradas frente a pruebas escritas cuesta diez minutos y encuentra incoherencias que, sin ese paso, aparecerían en forma de dos tareas terminadas que no encajan.

Paso 4. Construir tarea a tarea, con pruebas primero

Cada tarea se ejecuta por separado y deja dos ficheros: un **brief** con lo que hay que hacer y un **informe** con lo que se hizo. Dentro de cada una, el orden es siempre el mismo: se escribe la prueba, se comprueba que falla por el motivo correcto, se escribe el código mínimo que la hace pasar, y se refactoriza. El repositorio conserva veintiséis de esos ficheros de la fase 1.

La consecuencia se ve en las cifras: 5.534 líneas de pruebas frente a 5.671 de código. No es casualidad ni celo excesivo; es lo que sale cuando la prueba se escribe antes.

Paso 5. Revisar el código, y revisar la revisión

Al terminar cada tanda de tareas se genera un diff acumulado y se revisa buscando defectos reales, no estilo. El repositorio guarda diecinueve de esos diffs con nombres como `review-a617c3c..02b9d6f.diff`. Los hallazgos se corrigen en commits propios, y su rastro está en el historial:

- «fix: validar PATCH de términos, recalcular clave de deduplicación y no ocultar errores de carga»

- «Fix weak test assertion: verify exact shuffle output» — una prueba que pasaba sin comprobar nada útil
- «fix: exponer respuestas de repaso que se pierden tras agotar reintentos»
- «Reforzar la revisión de la Task 6: prueba de regresión real y orden seguro en la migración»

Ese último es interesante porque es una revisión *de la revisión*: alguien miró las correcciones y encontró que la prueba de regresión no ejercitaba de verdad el fallo.

Paso 6. Aislar el trabajo y fundirlo

Cada entrega vivió en su propia rama —fase-1-extraccion, colecciones-repaso— creada desde main con consentimiento explícito y con la referencia de la base anotada en el fichero de seguimiento. Dos entregas pasaron además por una *pull request* en GitHub. El fichero `.gitignore` tiene dos líneas que cuentan esta parte de la historia:

```
# worktrees locales de Claude Code (checkouts de otras sesiones)
.claude/worktrees/

# andamiaje de ejecución de planes (no forma parte del proyecto)
/.superpowers/
```

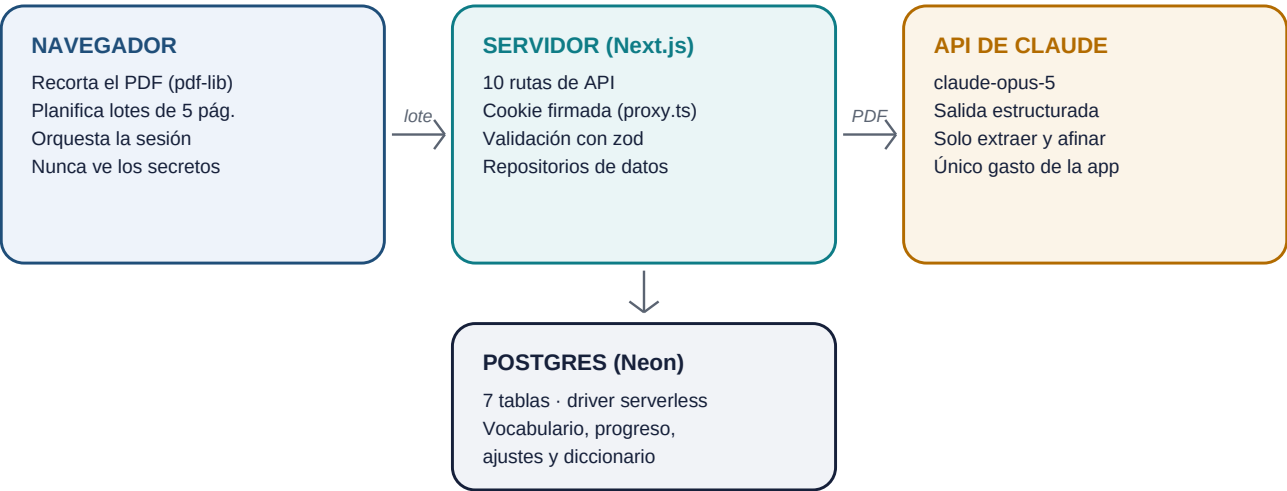
LO QUE APRENDÍ

El andamiaje de un proyecto —briefs, informes, diffs de revisión— es valiosísimo mientras se construye y ruido cuando ya está construido. Ignorarlo en git, en lugar de borrarlo o de commitarlo, deja lo mejor de ambos: está en el disco mientras haga falta y no ensucia la historia del proyecto.

SECCIÓN 6

La arquitectura, y por qué es así

Una sola aplicación Next.js con App Router, en TypeScript y con Tailwind, desplegada en Vercel: interfaz y rutas de servidor en el mismo proyecto y el mismo despliegue. No hay microservicios, ni un backend aparte, ni una cola de trabajos.



El PDF original no sale del equipo · Los secretos solo existen en el servidor

Las piezas y su papel

Pieza	Tecnología	Papel
Base de datos	Postgres en Neon, con Drizzle	Vocabulario y estado de repaso
Extracción	@anthropic-ai/sdk, claude-opus-5	Convierte páginas en términos
Recorte de PDF	pdf-lib, en el navegador	Aísla el rango de páginas antes de subirlo
Repetición espaciada	ts-fsrs	Calcula la próxima fecha de cada tarjeta
Acceso	Cookie de sesión firmada	Contraseña única, sin cuentas
Móvil	Manifiesto PWA	Instalable en la pantalla de inicio

Los límites de la plataforma, que condicionan el diseño

Tres números de la documentación de Vercel y de la API explican por qué la extracción funciona como funciona. No son detalles: son la razón de que exista el troceado en lotes.

- **60 segundos** es lo que Vercel deja correr a una función de servidor en el plan gratuito. Una extracción de treinta páginas no cabe. De ahí los lotes de cinco páginas, enviados en secuencia y guardados uno a uno: si el lote siete falla, los seis anteriores ya están en la base.
- **4,5 MB** es el tamaño máximo del cuerpo de una petición. Recortar el PDF en el navegador mantiene los envíos muy por debajo, y de paso resuelve la privacidad: el libro entero nunca sale del equipo.

- **32 MB y 600 páginas** es lo que acepta la API de Claude por petición: muy por encima de lo que se envía. El límite que manda es el de la plataforma, no el del modelo.

LO QUE APRENDÍ

Antes de diseñar un flujo largo conviene apuntar los tres o cuatro límites duros del sitio donde va a correr. Aquí, «60 segundos» no fue un problema que apareciera en producción: fue una línea de la especificación que hizo que el troceado en lotes estuviera en el plan desde el primer día.

El modelo de datos: siete tablas y una idea

La idea que ordena el esquema es que **el término y su progreso viven separados**. Corregir una traducción a mano no puede costar el historial de repaso de esa palabra.

Tabla	Qué guarda	Detalle que merece la pena
sources	Cada extracción: título, páginas, nivel, tokens y coste	Guarda el coste real en dólares de esa llamada
terms	El vocabulario: término, traducción, tipo, nivel, pista en inglés	Índice único sobre (término normalizado, pista), no sobre el término solo
term_occurrences	Dónde apareció cada término y con qué ejemplo	Un término repetido suma una aparición, no una tarjeta nueva
card_states	El estado FSRS de cada tarjeta	Una fila por término, con la clave primaria en term_id
review_logs	El histórico de respuestas	Índice único sobre answer_id: la idempotencia vive aquí
settings	Los tres ajustes del usuario	Tabla de una sola fila, con id por defecto a 1
dictionary_entries	Las 181.103 acepciones de Wikcionario	Sin índice único: la gracia es que un término tenga varias acepciones

SECCIÓN 7

Fase 1 — Extraer vocabulario de un PDF

El recorrido de una extracción

- 1 El usuario elige un PDF, un rango de páginas y un nivel del MCER en `/extraer`.
- 2 El **navegador** abre el PDF con `pdf-lib`, planifica los lotes de cinco páginas y, para cada lote, construye un PDF nuevo que solo contiene esas páginas.
- 3 Ese trozo se codifica en base64 y se envía a `POST /api/extract`.
- 4 El servidor llama a `claude-opus-5` pasándole el PDF como documento y un prompt construido a partir del nivel, y exige la respuesta en un formato fijo.
- 5 El resultado se guarda en Postgres dentro de una transacción, deduplicando por término normalizado, y se devuelven al navegador los términos, cuántos son nuevos, cuántos se fusionaron y el coste real.
- 6 El navegador pinta el progreso y pide el siguiente lote. Nunca dos a la vez.

Salida estructurada: pedirle al modelo una forma, no un texto

La pieza clave de la fase 1 es que a Claude no se le pide «devuélveme una lista»: se le pasa un esquema y se le obliga a rellenarlo. El esquema se declara una sola vez con zod y se convierte al formato que espera la API:

```
const response = await client.messages.parse({
  model: "claude-opus-5",
  max_tokens: 16000,
  thinking: { type: "adaptive" },
  messages: [{ role: "user", content: [
    { type: "document", source: { type: "base64",
      media_type: "application/pdf", data: params.pdfBase64 } },
    { type: "text", text: buildExtractionPrompt(level, pageStart, pageEnd) },
  ]}],
  output_config: { format: zodOutputFormat(extractionSchema) },
});
```

Eso elimina de golpe una familia entera de problemas: no hay que parsear texto libre, no hay que tolerar comillas raras ni listas con guiones, y TypeScript conoce el tipo del resultado sin que nadie lo escriba dos veces. Un mismo esquema de zod hace tres trabajos: instruye al modelo, valida la respuesta y genera el tipo.

Quedan dos casos que sí hay que tratar a mano, y ambos están en el código: que el modelo rechace la petición (`stop_reason === "refusal"`) y que no devuelva un resultado analizable. Los dos levantan un error con un mensaje en español, no el mensaje crudo de la librería.

El coste, medido y no estimado

Cada respuesta de la API trae cuántos tokens de entrada y de salida ha consumido. Con los precios del modelo, convertirlo en dinero son cuatro líneas:

```
/** Precios de claude-opus-5, en dólares por token. */
const INPUT_USD_PER_TOKEN = 5 / 1_000_000;
const OUTPUT_USD_PER_TOKEN = 25 / 1_000_000;

export function estimateCostUsd(usage) {
  return usage.input_tokens * INPUT_USD_PER_TOKEN
    + usage.output_tokens * OUTPUT_USD_PER_TOKEN;
}
```

El coste se guarda en la fila de la fuente y se enseña en la pantalla. La primera medición real, el 6 de septiembre, fue de **26 términos por unos 0,07 €**, es decir alrededor de 0,003 € por término: mil términos en la biblioteca costarían menos de 3 €. El diseño había estimado entre 15 y 30 céntimos por siete u ocho páginas; la realidad salió aproximadamente la mitad de cara, y hay un commit dedicado a sustituir la estimación por la medición.

LO QUE APRENDÍ

Cuando un producto llama a una API de pago, enseñar el coste real de cada operación en la propia interfaz cambia cómo se usa. No es una métrica de administrador: es información que el usuario necesita para decidir si extrae diez páginas o cincuenta. Y una estimación en un documento de diseño se sustituye por una medición en cuanto existe la primera.

Deduplicación: la misma palabra en dos libros

Un término que ya está en la biblioteca no crea una tarjeta nueva: suma una aparición. La clave de comparación es deliberadamente sencilla y vive en una función de una línea:

```
export function normalizeTerm(term: string): string {
  return term.normalize("NFC").trim().toLowerCase().replace(/\s+/g, " ");
}
```

Las cuatro operaciones importan. NFC es la que menos se ve y la más necesaria: la letra é puede llegar como un único carácter o como una e seguida de una tilde combinante, y sin normalizar la forma Unicode son dos cadenas distintas que se ven idénticas en pantalla. Hay un commit exclusivo para arreglar la prueba que decía comprobar esto y no lo comprobaba: «fix: prueba real de normalización Unicode».

Y hay una garantía que costó un commit propio: una traducción corregida a mano en la biblioteca **no se pierde** si el término vuelve a salir en una extracción posterior. La fusión respeta lo que el usuario escribió.

SECCIÓN 8

Fase 2 — Repasar con repetición espaciada

Qué es la repetición espaciada, en dos párrafos

La curva del olvido dice que lo aprendido se desvanece rápido si no se recupera. La repetición espaciada aprovecha lo contrario: cada vez que uno recuerda algo con esfuerzo, el recuerdo se consolida y aguanta más tiempo. La estrategia consiste en enseñar cada tarjeta justo antes de olvidarla, y en alargar el intervalo cada vez que se acierta.

El algoritmo elegido es **FSRS** (*Free Spaced Repetition Scheduler*), la generación posterior al clásico SM-2 de Anki. En lugar de un solo «factor de facilidad», modela cada tarjeta con tres variables —**estabilidad** (cuánto aguanta el recuerdo), **dificultad** (lo costoso que es ese material) y **retrievability** (la probabilidad de acertar ahora mismo)— y con ellas calcula el próximo intervalo. En Vocably lo implementa el paquete `ts-fsrs`.

Las cuatro respuestas y los cuatro estados

El usuario ve la palabra, la piensa, pulsa (o la barra espaciadora) para ver la traducción y el ejemplo, y valora con uno de cuatro botones —**Otra vez**, **Difícil**, **Bien**, **Fácil**, también accesibles con las teclas 1 a 4—. Esa valoración es lo único que come el algoritmo. De ahí salen los cuatro estados que FSRS escribe en `card_states.state`, y de esos estados sale, sin guardar nada nuevo, la idea de «colección»:

Estado	Significado	Colección
0 — Nueva	Nunca se ha respondido	No aprendidas
1 — Aprendiendo	En los pasos cortos iniciales	En curso
3 — Reaprendiendo	Se falló y ha vuelto a pasos cortos	En curso
2 — Repaso	Consolidada, con intervalo de días o meses	Aprendidas

Una palabra sube a «aprendidas» al acertarla y baja sola al fallarla. No hay ningún botón de «ya me la sé»: fue una decisión explícita de diseño, porque el estado ya lo sabe el algoritmo.

El puente entre dos mundos: `snake_case` y `camelCase`

Un detalle pequeño con una lección detrás. `ts-fsrs` usa `snake_case` (`elapsed_days`, `last_review`) y el esquema de la base usa `camelCase`. En vez de esparcir conversiones por todo el código, hay un único fichero con dos funciones —`toFsrsCard` y `fromFsrsCard`— que traducen en las dos direcciones. Toda la incompatibilidad de la librería está encerrada en veinte líneas.

LO QUE APRENDÍ

Cuando una librería externa impone convenciones que chocan con las del proyecto, la solución barata es adaptarse a ella en todas partes y la solución correcta es aislarla en un adaptador. El día que se cambie de librería, solo hay un fichero que reescribir.

Idempotencia: cerrar la app a mitad de sesión

El repaso ocurre en el móvil, en huecos muertos: en el metro, en una cola. La app se cierra a mitad de sesión constantemente. Si una respuesta enviada se reintenta al volver, no puede contarse dos veces.

La solución es que cada respuesta viaja con un identificador propio, el `answerId`, generado por el cliente, y que la tabla del histórico tiene un **índice único** sobre esa columna. Repetir la misma respuesta no duplica el registro ni vuelve a mover la tarjeta: la base de datos la rechaza sola. Y como el resultado se escribe en el momento de responder, cerrar la app y reabrirla no hace reaparecer una tarjeta ya respondida.

UNA SUTILEZA QUE PARECE UN FALLO Y NO LO ES

Si una tarjeta valorada «Otra vez» vuelve a salir a los pocos minutos, no es que el sistema haya olvidado la respuesta: es un paso de aprendizaje corto de FSRS, funcionando exactamente como debe. Está documentado en el README precisamente porque es el tipo de comportamiento que un usuario reporta como error.

¿Cuándo empieza «hoy»?

El tope de palabras nuevas es diario, así que hace falta una definición de día. La respuesta obvia —medianoche UTC— está mal: en verano, en España, eso son las dos de la madrugada, con lo que un repaso a la una contaría contra el cupo del día anterior. El día de la aplicación empieza a **medianoche en Madrid**.

Calcularlo bien tiene un pliegue: los dos domingos del año en que cambia la hora, el desfase del instante actual no es el mismo que el desfase de la medianoche de ese día. El 29 de marzo a mediodía Madrid va en +2, pero su medianoche fue todavía +1. La función hace por eso **dos pasadas**: calcula una medianoche aproximada con el desfase de ahora y la recoloca con el desfase de esa medianoche candidata.

Y una decisión de diseño con consecuencias en las pruebas: el instante «ahora» nunca se obtiene con `new Date()` dentro de la lógica; se recibe como parámetro. Así las pruebas mandan sobre el reloj y se puede comprobar el comportamiento del cambio de hora sin esperar a marzo.

Los tres ajustes

Ajuste	Por defecto	Qué hace
<code>newCardsPerDay</code>	20	Cuántas palabras nuevas entran al día. Es diario: se cuentan las introducciones ya registradas hoy y se resta.
<code>sessionSize</code>	0	Cuántas palabras entran en total en la sesión. 0 significa «las que toquen hoy» . Cualquier otro número manda incluso por encima del tope diario.
<code>sessionMode</code>	mezcla	De qué colección salen: no-aprendidas, aprendidas o mezcla.

Con qué criterio se guarda cada uno es, en sí mismo, una decisión de producto: `newCardsPerDay` se guarda al salir del campo, porque es un ajuste permanente; los otros dos son la elección *de esta sesión* y solo se guardan si el usuario marca «Recordar esta elección».

SECCIÓN 9

Fase 3 — El diccionario propio: 181.103 entradas por 0 €

Hasta aquí, el vocabulario solo podía entrar por una puerta: extraerlo de un PDF pagando la API. Si uno oye una palabra en clase o la lee en un artículo, no tiene dónde meterla. La fase 3 abre la segunda puerta, y la abre **gratis**.

De dónde salen 181.103 entradas

De un volcado de **Wikcionario en inglés**, distribuido por kaikki.org en formato JSONL: un objeto JSON por línea, una línea por acepción, con su categoría gramatical, su definición en inglés, ejemplos y a veces traducciones. Licencia libre y descarga directa.

El volcado crudo es enorme, así que se filtra en dos ejes: por **categoría gramatical** —sustantivo, verbo, adjetivo, adverbio, expresión, verbo frasal e interjección— y por **frecuencia**, quedándose con las 50.000 palabras sueltas más frecuentes del inglés. Las expresiones de varias palabras (verbos frasales, *idioms*) entran **todas, sin filtro de frecuencia**, por una razón práctica: no aparecen en ninguna lista de frecuencia, así que filtrarlas por ese criterio las eliminaría por completo. Resultado: 181.103 entradas, unos 36 MB en Postgres.

POR QUÉ EL DICCIONARIO VIVE EN LA BASE DE DATOS

La tentación era guardar el fichero aparte e inventar un sistema de trozos para consultarlo. La decisión fue meterlo en Postgres sin más: 36 MB sobre los 500 MB gratuitos de Neon caben de sobra, y así el diccionario se consulta con la misma conexión, el mismo índice y las mismas pruebas que todo lo demás. La complejidad que no se construye es la única que nunca falla.

La escalera de cuatro escalones

Una búsqueda recorre cuatro fuentes y se para en la primera que responde. El orden no es casual: va de lo instantáneo y gratis a lo lento y caro.

#	Dónde busca	Tarda	Cuesta
1	La biblioteca del propio usuario	instantáneo	0 €
2	La tabla del diccionario (Wikcionario)	instantáneo	0 €
3	El traductor MyMemory, solo si a la entrada le falta el español	~1 s (5 s como máximo)	0 €
4	Claude, solo si se pulsa «Afinar con IA»	~3 s	unos céntimos

Y una separación deliberada en el código: los escalones 1 a 3 los resuelve GET /api/diccionario, que **no llama a Claude nunca**. El escalón 4 vive en una ruta aparte, POST /api/diccionario/afinar, precisamente para que se vea de un vistazo que ninguna otra ruta toca la API de pago.

LO QUE APRENDÍ

Separar en rutas distintas lo gratuito de lo que cuesta dinero no es una cuestión de organización: es una propiedad verificable. Cualquiera puede abrir el fichero y comprobar que la búsqueda no puede facturar, sin leer la lógica entera.

El traductor, y la lección de cuándo NO cachear

Wikcionario es fortísimo en definiciones en inglés y flojísimo en traducciones: solo el **1,8 %** de lo cargado trae español. El resto lo traduce MyMemory, un servicio gratuito. Dos detalles medidos a mano que cambiaron el diseño:

- **Se manda el término solo, nunca pegado a su definición.** Mandarlos juntos para desambiguar parece buena idea y no lo es: «*come across: to give an impression*» vuelve como «*venir a través: para dar una impresión*», partiendo el verbo frasal en dos.
- **La traducción solo se guarda cuando el término tiene una única acepción sin español.** Si hay varias —*bank* tiene siete entradas en Wikcionario, una por etimología— la traducción se enseña en todas pero no se cachea en ninguna.

El motivo del segundo punto es la mejor lección de esta fase: al traductor se le manda el término suelto, así que lo que vuelve es la traducción *de la palabra*, no la de una acepción concreta. Escribir «banco» en la acepción de *orilla de un río* y darla por buena la dejaría mal **para siempre**, porque una fila cacheada no se vuelve a consultar. Como está escrito en el código: «Preguntar otra vez es gratis; equivocarse en la base, no».

LA CAÍDA Y EL COLGADO NO SON EL MISMO FALLO

El código ya capturaba las excepciones del traductor, así que un servicio *caído* estaba cubierto. Pero uno *colgado* —que acepta la conexión y no responde nunca— dejaba esperando a la búsqueda entera, que para entonces ya tenía el significado y el ejemplo listos para enseñar. La solución fueron cinco segundos de plazo con `AbortSignal.timeout`. Un `try/catch` no protege del tiempo.

La pista en inglés: guardar *bank* dos veces

Una palabra puede tener más de un significado que interesa guardar por separado: *bank* es la orilla de un río y también donde se guarda el dinero. Para permitirlo hizo falta cambiar el esquema: se añadió una columna `sense_hint` —el significado en inglés— y el índice único de la tabla de términos pasó de ser sobre el término normalizado a serlo sobre **(término normalizado, pista)**.

Esa pista aparece luego en la cara delantera de la tarjeta de repaso, pequeña y recortada a una línea: dice lo justo para distinguir *bank* (orilla) de *bank* (banco) sin chivar la traducción al español antes de tiempo. Hay un commit dedicado a quitarla de donde estorbaba: «No repetir la glosa inglesa en la cara delantera de la tarjeta».

Cargar el diccionario: un paso manual, y a propósito

```
DATABASE_URL='...' npm run cargar:diccionario -- ~/Vocably-diccionario/dicc_todo.jsonl.gz
```

El cargador vacía la tabla y la vuelve a llenar entera. Para 181.103 filas es más simple y más seguro que fusionar fila a fila, y el diccionario es material de consulta, no datos del usuario: no hay nada que perder al recargarlo. Es un paso que una persona ejecuta **una vez**, igual que la creación de las tablas; no forma parte del arranque de la app ni se repite en cada despliegue, porque el diccionario no cambia.

El commit que ordenó esta parte tiene un título que resume una regla general de dependencias: «No leer el diccionario antes de tener dónde meterlo».

SECCIÓN 10

Fase 4 — Elegir qué y cuánto repasar

El problema: un ajuste que solo aparecía cuando no servía

Existía un ajuste para limitar los repasos por sesión, pero solo se pintaba en dos pantallas: cuando **no** tocaba ninguna tarjeta y cuando **ya** habías terminado. Es decir, en los dos únicos momentos en los que no sirve de nada. Si hoy había palabras pendientes, no había dónde tocarlo.

La solución fue una **pantalla previa**: antes de ver la primera tarjeta se elige de qué colección salen las palabras y cuántas entran.

Cuatro decisiones de producto, todas del usuario

- 1 **Pantalla previa cada vez, con memoria.** Sale siempre, rellena con lo que se guardó la última vez. Se puede cambiar solo por hoy, o marcar que se recuerde.
- 2 **Las colecciones las decide el algoritmo**, no un botón. Una palabra sube a «aprendidas» al acertarla y baja sola al fallarla.
- 3 **Elegir una colección puede adelantar palabras que aún no vencían**, y lo que se responda cuenta como cualquier otro repaso.
- 4 **El número elegido manda sobre el tope diario de palabras nuevas.** Si se piden 30, da 30, aunque el tope solo dejara entrar 5.

Sortear, no ordenar

Cuando vencen más palabras de las que caben en la sesión, hay que elegir cuáles entran. La tentación es ordenar por antigüedad o por identificador; el problema es que entonces las mismas palabras quedan siempre al final de la cola y no se repasan nunca. La solución es **sortearlas** con una baraja de Fisher-Yates.

Dos detalles del código merecen mención. El primero: la fuente de azar **se inyecta** en vez de llamar a `Math.random` directamente, porque el sorteo decide qué ve el usuario y una prueba que no pueda fijar el azar solo podría comprobar longitudes, no que el sorteo sea correcto. El segundo: hay un ajuste defensivo para un generador inyectado que devuelva exactamente 1 —`Math.random` nunca lo hace, pero uno de prueba sí— que sin él sacaría el índice fuera del array.

```
export function barajar<T>(elementos, aleatorio = Math.random): T[] {
  const copia = [...elementos];
  for (let i = copia.length - 1; i > 0; i--) {
    const j = Math.min(i, Math.floor(aleatorio() * (i + 1)));
    [copia[i], copia[j]] = [copia[j], copia[i]];
  }
  return copia;
}
```

Y dos garantías que el sorteo respeta: **las palabras en curso nunca se sortean y van siempre las primeras** —son las que se acaban de fallar y el algoritmo quiere volver a preguntar en minutos—, y **lo que queda fuera no se pierde**: sigue vencido y entra en la sesión siguiente. Al terminar, la pantalla dice cuántos repasos quedaron fuera del límite y ofrece seguir.

LO QUE APRENDÍ

Un límite silencioso es una trampa. Un tope por debajo del ritmo diario acumula atrasos, y sin un aviso al final de la sesión lo haría en silencio hasta que un día aparecen trescientas tarjetas pendientes. Cuando un producto recorta algo por el usuario, tiene que decirlo.

Lo que se guarda y lo que no

El detalle más fino de esta fase, y el que costó un commit propio: cuando el modo guardado se queda sin material hoy, la pantalla se cae al primero que sí lo tenga. Eso **no** debe sobrescribir la preferencia del usuario, porque es una circunstancia del día, no una elección. El commit se llama «No guardar como elección del usuario el modo al que cae la pantalla».

Y una consecuencia obligada del cambio: como el número nuevo es un total de sesión y el viejo `reviewsPerSession` limitaba solo los repasos, **los dos no podían convivir**. El viejo se sustituyó, lo que obligó a la migración más delicada del proyecto (sección 13).

SECCIÓN 11

Catálogo de herramientas descubiertas

Veinticuatro piezas —librerías, servicios, formatos y métodos— que este proyecto puso en las manos por primera vez. De cada una: qué es, para qué sirvió aquí, y qué me llevo.

A. Framework y lenguaje

01 Next.js 16 (App Router)

Framework de React · el esqueleto de todo

Qué es. Un framework que junta en un solo proyecto la interfaz de React y el servidor. El App Router organiza la aplicación por carpetas: una carpeta es una ruta, y dentro conviven la página y sus rutas de API.

Aquí. Las seis pantallas y las diez rutas de API viven en el mismo despliegue. No hay backend aparte ni configuración de servidor.

Lo que aprendí. La frontera entre «esto corre en el navegador» y «esto corre en el servidor» deja de ser una cuestión de carpetas y pasa a ser una decisión por fichero. Entenderla bien es lo que permite que la clave de la API solo exista en el servidor sin escribir una línea de configuración.

02 React 19

Librería de interfaz

Qué es. La librería sobre la que se pinta todo: componentes que reciben datos y devuelven la pantalla correspondiente.

Aquí. La sesión de repaso, el formulario de extracción con su progreso por lotes, la tabla editable de la biblioteca y el buscador del diccionario.

Lo que aprendí. La interfaz más difícil del proyecto no fue la más bonita, sino la que tiene *estados*: cargando, error, sin material, a mitad de sesión, terminado. Enumerarlos antes de escribir el componente ahorra la mitad de los fallos.

03 TypeScript

Lenguaje · tipos sobre JavaScript

Qué es. JavaScript con un sistema de tipos que se comprueba antes de ejecutar nada.

Aquí. Todo el proyecto. Los tipos de la respuesta del modelo salen del mismo esquema que la valida, así que no hay forma de que se desincronicen.

Lo que aprendí. El valor de los tipos no está en atrapar erratas: está en que un cambio de esquema en la base de datos aparezca inmediatamente como veinte errores en los sitios exactos que hay que tocar. `npx tsc --noEmit` se convirtió en un paso más de la verificación.

04 Tailwind CSS 4

Estilos · utilidades en el marcado

Qué es. Un sistema de estilos donde en lugar de escribir CSS se componen clases pequeñas directamente en el marcado.

Aquí. Todo el diseño visual, incluidos los dos rediseños completos —extracción y biblioteca— del segundo día.

Lo que aprendí. Rediseñar una pantalla entera deja de dar miedo cuando los estilos viven junto al marcado: no hay una hoja global que pueda romper otra pantalla. A cambio, hace falta disciplina para que los componentes base —botón, campo, tarjeta— existan de verdad y no se reinventen en cada sitio.

B. Datos

05 Neon (Postgres serverless)

Base de datos · plan gratuito

Qué es. Postgres alojado, con un plan gratuito de 500 MB, pensado para entornos sin servidor: la conexión se abre y se cierra por petición.

Aquí. Las siete tablas, incluidas las 181.103 filas del diccionario (unos 36 MB de esos 500).

Lo que aprendí. Que existan 500 MB gratis cambia decisiones de arquitectura. La alternativa a meter el diccionario en la base era inventar un sistema de trozos en ficheros: más código, más fallos y ninguna ventaja mientras quepa.

06 Drizzle ORM

Acceso a datos · SQL con tipos

Qué es. Una capa fina sobre SQL: el esquema se declara en TypeScript y las consultas se escriben casi como SQL, pero con los tipos comprobados.

Aquí. Todo el acceso a la base, repartido en cinco repositorios (extracción, términos, repaso, ajustes y diccionario).

Lo que aprendí. Un detalle que costó entender: el proyecto usa el driver neon-serverless con Pool, no neon-http, y **no son intercambiables**. Guardar una extracción necesita una transacción real para fusionar términos de forma atómica, y eso solo lo da el primero.

07 drizzle-kit

Migraciones · generar y aplicar cambios de esquema

Qué es. La herramienta que compara el esquema declarado con la base real y genera los cambios.

Aquí. Seis migraciones, de la creación inicial de tablas al cambio de ajustes de la cuarta fase.

Lo que aprendí. La lección más cara del proyecto está aquí: push y migrate no son lo mismo. push **no ejecuta los ficheros .sql**: compara y propone él mismo las sentencias, preguntando por cada una que pueda perder datos. Aceptar sin leer puede convertir dos columnas nuevas en un «renombrado» que deja el tipo equivocado. Se cuenta entero en la sección 13.

08 PGlite

Pruebas · Postgres de verdad, en memoria

Qué es. Postgres compilado para correr dentro del propio proceso, sin instalar ni levantar un servidor.

Aquí. Todas las pruebas que tocan datos: la carga del diccionario, la deduplicación, la cola del repaso, la idempotencia de las respuestas.

Lo que aprendí. Es la herramienta que más cambió la forma de probar. No hay que elegir entre «pruebas rápidas con la base falseada» y «pruebas lentas contra una base real»: se prueban las consultas de verdad, con SQL de verdad, en cuarenta segundos y sin tocar nada real.

C. Inteligencia artificial

09 @anthropic-ai/sdk

Cliente oficial de la API de Claude

Qué es. La librería que habla con la API: mensajes, documentos adjuntos, uso de tokens.

Aquí. La extracción de vocabulario y el botón opcional de afinar una traducción.

Lo que aprendí. Un PDF se manda como un bloque de contenido de tipo `document` en base64, junto al texto del prompt, y el modelo lee texto e imágenes de la página en una sola pasada. Eso hace que los PDF escaneados funcionen sin añadir OCR.

10 Salida estructurada

Patrón · pedir una forma, no un texto

Qué es. Un modo de la API en el que se declara el esquema exacto de la respuesta y el modelo está obligado a rellenarlo, en lugar de devolver texto libre.

Aquí. El resultado de cada lote: una lista de términos con tipo, traducción, contexto y ejemplo.

Lo que aprendí. Es la diferencia entre integrar un modelo y pelearse con él. Desaparece el parseo frágil, desaparecen los formatos inventados y el tipo de TypeScript sale del mismo esquema. Quedan solo dos casos que tratar a mano: que el modelo rechace la petición y que no devuelva nada analizable.

11 zod

Validación · un esquema, tres trabajos

Qué es. Una librería para declarar la forma de un dato y validarlo en tiempo de ejecución, derivando además el tipo de TypeScript.

Aquí. El esquema de la extracción, y la validación de los cuerpos de las rutas de API.

Lo que aprendí. El mismo esquema instruye al modelo, valida su respuesta y genera el tipo. Una sola fuente de verdad para tres cosas que, sin él, se escriben por separado y se desincronizan a la tercera semana.

D. Pruebas y calidad

12 Vitest

Ejecutor de pruebas

Qué es. Un ejecutor de pruebas rápido, con la misma interfaz que Jest pero más liviano.

Aquí. Las 440 pruebas del proyecto, en 42 ficheros, en 43 segundos.

Lo que aprendí. Un detalle diminuto que quedó documentado para que nadie lo «arregle»: el fichero de configuración es `vitest.config.mts`, con extensión `.mts` y no `.ts`, a propósito, porque así la salida de las pruebas queda limpia de avisos.

13 ESLint 9 (configuración plana)

Análisis estático

Qué es. El analizador de estilo y errores comunes de JavaScript, con el formato de configuración nuevo en un solo fichero.

Aquí. Todo el proyecto, junto a la comprobación de tipos, antes de cada fusión.

Lo que aprendí. Sirve de red bajo los cambios grandes: hay un commit llamado «Corregir la regresión de lint al refrescar la biblioteca tras editar» que existe justo porque el analizador vio lo que la prueba no cubría.

E. Librerías de dominio

14 pdf-lib

PDF · manipulación en el navegador

Qué es. Una librería que lee y escribe PDF en JavaScript, y que funciona igual en el navegador que en el servidor.

Aquí. Recortar el rango de páginas y construir un PDF nuevo con solo esas páginas, **antes** de enviar nada.

Lo que aprendí. Mover ese recorte al navegador resolvió tres problemas de una vez: privacidad (el libro no sale del equipo), tamaño (el envío cabe en el límite de 4,5 MB) y coste (no se paga por procesar páginas que no interesan).

15 ts-fsrs

Repetición espaciada · el algoritmo FSRS

Qué es. La implementación en TypeScript de FSRS, el planificador de repetición espaciada que sucede al clásico SM-2 de Anki.

Aquí. Todo el cálculo de cuándo vuelve a tocar cada palabra, a partir de las cuatro valoraciones.

Lo que aprendí. No hay que inventar un algoritmo de memoria: hay que entender el que ya existe. Lo que sí hay que escribir es el *punte* entre sus convenciones y las del proyecto, y encerrarlo en un solo fichero.

16 tsx

Ejecutar TypeScript directamente

Qué es. Un lanzador que ejecuta ficheros TypeScript sin compilarlos antes.

Aquí. El cargador del diccionario: `npm run cargar:diccionario`.

Lo que aprendí. Los scripts de mantenimiento no tienen por qué vivir fuera del proyecto ni en otro lenguaje. Escribirlos en el mismo TypeScript significa que reutilizan el esquema, los tipos y las mismas funciones que la aplicación.

F. Datos externos gratuitos

17 Wikcionario vía kaikki.org (JSONL)

Fuente de datos · licencia libre

Qué es. Un volcado completo y estructurado de Wikcionario, distribuido como JSONL: un objeto JSON por línea, una línea por acepción.

Aquí. Las 181.103 entradas del diccionario, filtradas por categoría gramatical y frecuencia.

Lo que aprendí. El formato JSONL es la mitad del descubrimiento: se procesa línea a línea, sin cargar el fichero entero en memoria, y comprimido con gzip ocupa una fracción. La otra mitad es que existen volcados libres y buenos de recursos que uno da por hecho que hay que pagar.

18 MyMemory

Traducción automática gratuita

Qué es. Un servicio de traducción con cuota anónima —unos 5.000 caracteres al día, y un término son unos diez— que no pide clave.

Aquí. Traducir al español las entradas del diccionario que no lo traen, que son el 98,2 %.

Lo que aprendí. Se eligió después de comprobar a mano que las instancias públicas de las alternativas estaban caídas. La lección es de método: cuando se depende de un servicio de terceros gratuito, hay que probarlo *ese día*, y hay que decidir de antemano qué hace la aplicación cuando no responda.

G. Plataforma y web moderna

19 Vercel

Despliegue continuo

Qué es. La plataforma donde vive la aplicación. Cada empujón a la rama principal despliega solo.

Aquí. El despliegue de las cuatro fases, con las cuatro variables de entorno configuradas en el proyecto.

Lo que aprendí. Sus límites son parte del diseño, no una sorpresa: 60 segundos por función y 4,5 MB de cuerpo de petición. Apuntarlos en la especificación hizo que el troceado en lotes estuviera previsto desde el primer día.

20 Manifiesto de aplicación web (PWA)

Móvil · instalar sin tienda de apps

Qué es. Un fichero que describe la aplicación —nombre, icono, modo de presentación— y permite añadirla a la pantalla de inicio del móvil y abrirla sin la barra del navegador.

Aquí. `app/manifest.ts`, con modo `standalone` y, el detalle que más se nota, `start_url` apuntando a `/repaso`.

Lo que aprendí. Una web bien hecha se instala en el móvil con un fichero de veinte líneas: no hace falta una app nativa ni pasar por una tienda. Y elegir bien la pantalla de inicio —el repaso, no la extracción— cambia por completo para qué se abre la app.

21 APIs estándar modernas

Plataforma · lo que ya viene incluido

Qué es. Capacidades que hoy trae el propio lenguaje o el navegador y que antes exigían una librería: `AbortSignal.timeout` para cortar una petición, `Intl.DateTimeFormat` para trabajar con zonas horarias, `String.normalize` para las formas Unicode y `node:crypto` para firmar la cookie de sesión.

Aquí. El plazo del traductor, la medianoche de Madrid, la clave de deduplicación y la sesión.

Lo que aprendí. Cuatro dependencias que no hizo falta instalar. Antes de añadir un paquete conviene mirar si la plataforma ya lo trae: las zonas horarias, sobre todo, tienen fama de necesitar una librería grande y no la necesitan.

H. Método y herramientas de trabajo

22 Claude Code con «skills»

Agente de programación · flujo estructurado

Qué es. Un agente que trabaja dentro del repositorio y al que se le pueden dar *skills*: procedimientos escritos que fijan cómo abordar cada tipo de tarea —explorar el diseño antes de programar, escribir un plan, construir con pruebas primero, pedir revisión, verificar antes de declarar algo terminado—.

Aquí. Las cuatro entregas completas, del documento de diseño a la fusión.

Lo que aprendí. Un agente sin método produce código plausible; un agente con un plan verificado produce código que encaja. La diferencia no la marca el modelo: la marca el procedimiento, y ese procedimiento se puede escribir, versionar y reutilizar en el siguiente proyecto.

23 Worktrees de git

Control de versiones · varios frentes a la vez

Qué es. Un mecanismo de git para tener varias copias de trabajo del mismo repositorio, cada una en su rama, sin cambiar de rama ni guardar cambios a medias.

Aquí. Aislar el trabajo de cada fase, con una entrada propia en `.gitignore` para los checkouts de otras sesiones.

Lo que aprendí. Elimina el ritual de `git stash` cada vez que hay que mirar otra cosa. Cada línea de trabajo tiene su carpeta, sus dependencias instaladas y su servidor de desarrollo.

24 Revisión de código sistemática

Calidad · buscar defectos, no estilo

Qué es. Revisar cada tanda de trabajo contra un diff acumulado, con el encargo explícito de encontrar defectos reales —no opiniones de formato— y corregirlos en commits propios.

Aquí. Diecinueve diffs de revisión guardados solo en la primera fase, y una decena de commits que empiezan por «fix:» o por «Arreglar».

Lo que aprendí. Lo más valioso que encontraron las revisiones no fueron fallos del código, sino **pruebas que pasaban sin comprobar nada**. Una suite verde puede ser una suite ciega, y solo se descubre leyendo las aserciones una por una.

SECCIÓN 12

Conceptos de ingeniería que hubo que entender

Estas once ideas no son de este proyecto: son de cualquier proyecto. Aquí aparecieron con nombre y apellidos, y cada una dejó código que se puede señalar.

1. Normalización Unicode

Dos cadenas que se ven idénticas en pantalla pueden ser distintas para el ordenador. La letra é puede estar codificada como un único carácter o como una e seguida de una tilde combinante. Comparar sin normalizar produce duplicados invisibles: dos filas «iguales» que la base de datos considera diferentes.

La solución cabe en una llamada —`term.normalize("NFC")`— pero hay que saber que existe. En este proyecto está en la primera línea de la función que decide si dos términos son el mismo.

2. Idempotencia

Una operación es idempotente si repetirla da el mismo resultado que hacerla una vez. En cuanto una aplicación se usa en un móvil con mala cobertura, deja de ser un lujo: los reintentos son inevitables y una respuesta contada dos veces corrompe el histórico.

El patrón concreto: el cliente genera un identificador único por operación, lo manda con la petición, y el servidor pone un **índice único** sobre esa columna. La garantía no la da el código de la ruta —que puede tener carreras— sino la base de datos.

3. Transacciones y atomicidad

Guardar una extracción hace varias cosas: crear la fuente, insertar términos nuevos, fusionar los que ya existían y anotar las apariciones. O pasan todas o no pasa ninguna; a medias, la biblioteca queda incoherente.

Esto tuvo una consecuencia práctica en la elección de herramienta: obligó a usar el driver de Postgres que soporta transacciones reales en lugar del más simple. Hay un commit de la fase del diccionario dedicado exactamente a esto: «Arreglar: envolver carga del diccionario en transacción para atomicidad».

4. Índices únicos compuestos

Un índice único no solo acelera: **define qué significa «el mismo»** en el modelo de datos. Cuando el proyecto decidió permitir guardar *bank* como orilla y como banco, el cambio de fondo no fue de interfaz: fue mover el índice único de (término) a (término, pista).

Y el diccionario ilustra lo contrario: la tabla de acepciones **no lleva** índice único, y hay un comentario en el esquema que lo explica, porque la gracia es justamente que un término tenga siete filas.

5. Inyectar el reloj, el azar y la red

Tres cosas que hacen imposible probar un código si se llaman directamente: `new Date()`, `Math.random()` y `fetch()`. Las tres se reciben en este proyecto como parámetros con un valor por defecto.

El beneficio se ve en lo que se puede probar: el cambio de hora de marzo sin esperar a marzo, el sorteo exacto de la cola y no solo su longitud, y un traductor colgado sin esperar cinco segundos por prueba. El coste es un parámetro más en tres firmas.

6. Un try/catch no protege del tiempo

Capturar excepciones cubre el servicio que falla, no el que no contesta. Un servicio colgado acepta la conexión y deja la petición esperando indefinidamente, y con ella toda la pantalla.

Todo servicio externo necesita un plazo explícito. Aquí son cinco segundos con `AbortSignal.timeout`, y el fallo devuelve una lista vacía en lugar de una excepción, para que la caída de un tercero no tumbe una búsqueda que ya tenía el significado y el ejemplo listos.

7. El orden de las sentencias de una migración importa

Una migración puede fallar a mitad. Lo que ocurra entonces depende del orden en que estén escritas las sentencias, y ese orden se puede elegir para que el fallo sea benigno.

Dos migraciones de este proyecto llevan un comentario explicando su orden. Una pone los `ADD COLUMN` primero y el `DROP COLUMN` al final, «para que un fallo a mitad deje la tabla con columnas de más y no con ajustes de menos». Otra coloca el borrado del índice justo antes del que lo sustituye, «para que un fallo a mitad deje el índice viejo en pie en vez de dejar la tabla sin ningún índice único».

8. Cachear es apostar a que el dato no cambiará

Una caché convierte un dato incierto en un dato definitivo, porque una fila cacheada no se vuelve a consultar. Por eso la pregunta correcta antes de cachear no es «¿cuánto ahorro?» sino «¿qué pasa si lo que guardo está mal?».

En el diccionario, la traducción se guarda solo cuando el término tiene una única acepción sin español, porque solo ahí la respuesta es inequívocamente de esa acepción. En los demás casos se enseña y no se guarda.

9. No todos los entornos de ejecución son iguales

La protección de rutas de esta aplicación vive en un fichero llamado `proxy.ts` y no en el `middleware.ts` habitual. La razón está documentada para que nadie lo «arregle»: en esta versión del framework, el middleware corre siempre en un entorno restringido que no soporta el módulo de criptografía de Node, y la firma de la sesión lo necesita. Renombrarlo rompería el acceso.

La lección general: «JavaScript» no es un entorno; es varios, con APIs distintas. Antes de colocar código, hay que saber dónde se va a ejecutar.

10. Degradar con elegancia

Cuando una pieza opcional falla, el producto debe seguir siendo útil. Si el traductor no responde, la ficha del diccionario sale igual sin español y quedan dos salidas: escribir la traducción a mano —gratis, y es la que manda— o pulsar el botón de afinar, que cuesta unos céntimos.

Compárese con la alternativa fácil: enseñar «error al traducir» y no dejar guardar. El mismo fallo, dos productos distintos.

11. Limitaciones aceptadas, escritas

El README dedica un apartado a decir que la comprobación de la contraseña es una comparación normal de cadenas —no de tiempo constante— y que no hay límite de intentos. Es una debilidad real, aceptada a cambio de no añadir estado ni dependencias en una aplicación de un solo usuario, y que obliga a que la contraseña sea larga y aleatoria.

Escribir una limitación conocida no es confesar un fallo: es la diferencia entre una decisión y un descuido. Lo mismo pasa con el aviso de que si falta el secreto de sesión, el arranque falla con un error genérico de Node y no con un mensaje claro.

SECCIÓN 13

Errores cometidos y lo que enseñaron

Un historial de 112 commits con una decena que empiezan por «fix:» o «Arreglar» es un historial honesto. Estos son los siete tropiezos que dejaron algo aprovechable.

1. La migración que se podía aceptar mal

El síntoma. La cuarta fase substituyó un ajuste por dos nuevos: había que añadir dos columnas y borrar una, todo a la vez.

La causa. La herramienta de migraciones, al ver tres cambios simultáneos, puede ofrecer interpretarlo como un **renombrado**. Aceptar ese renombrado deja la columna nueva con el tipo equivocado —entero donde el esquema espera texto—, sin la otra columna, y hace que la lectura de ajustes falle en *todas* las peticiones.

La lección. Hay que leer lo que la herramienta pregunta en vez de ir aceptando. Y hay que dejar por escrito cuál es la respuesta correcta: el README incluye las consultas SQL exactas para verificar, después de migrar, que las columnas existen con el tipo correcto y que la vieja ya no está.

Commit: 0005_organic_scorpion.sql

2. «Estás al día» con la biblioteca a medio aprender

El síntoma. La pantalla del repaso felicitaba al usuario y daba la sesión por terminada cuando aún quedaban palabras sin aprender.

La causa. El recuento solo miraba lo que vencía hoy, no lo que la propia sesión había dejado fuera por el límite. Con un tamaño de sesión bajo, la app recortaba y luego decía que no quedaba nada.

La lección. Un mensaje de «has terminado» es una afirmación sobre el estado del mundo, y hay que comprobarla igual que se comprueba un cálculo. Dos commits consecutivos se dedicaron a esto: contar como pendiente lo que la sesión deja fuera, y no llamar vacía a una biblioteca llena.

Commit: 9bb28ac · c12b267

3. El control de ajustes que desaparecía

El síntoma. El campo del tope de tarjetas nuevas se quedaba deshabilitado, y el bloque de ajustes desaparecía justo al terminar la sesión, que es cuando uno querría tocarlo.

La causa. Dos estados de la interfaz que nadie había enumerado: «cargando» y «sesión terminada». La lógica era correcta; la pantalla, no.

La lección. Es el hallazgo más incómodo del proyecto y está escrito en el README: **estos dos fallos solo aparecieron conduciendo un navegador a mano**. Las pruebas corren en Node, sin navegador simulado, así que ninguna puede pulsar un botón ni comprobar qué se pinta. 440 pruebas verdes no dicen nada sobre si un campo está habilitado.

Commit: 7c5e417 · ef1f42c

4. La prueba que pasaba sin comprobar nada

El síntoma. Una prueba del sorteo de la cola estaba en verde y no verificaba el resultado del sorteo, solo que la lista tuviera la longitud correcta.

La causa. Al no poder fijar la fuente de azar, la prueba no tenía nada estable que afirmar, así que afirmaba lo único posible: algo trivial.

La lección. Una aserción débil es peor que ninguna prueba, porque da confianza falsa. La corrección fue doble: inyectar el azar para poder predecirlo, y reescribir la aserción para comprobar la permutación exacta.

Commit: 00a189e — «Fix weak test assertion: verify exact shuffle output»

5. Las cinco pantallas rotas de la pantalla previa

El síntoma. Al construir la pantalla previa al repaso, cinco de sus estados quedaron mal a la vez.

La causa. Una pantalla nueva multiplica los estados posibles: sin material hoy, con material solo en una colección, con la elección guardada imposible de cumplir, a mitad de carga, con error de red. Cada combinación es una pantalla distinta.

La lección. El coste de una pantalla no se mide en componentes, sino en estados. Enumerarlos antes de escribirla es más barato que descubrirlos de uno en uno.

Commit: 81d23d6

6. Cachear una traducción ambigua

El síntoma. El diseño original decía, sin matices, que la traducción del servicio externo se guardaría en la fila del diccionario.

La causa. Al construirlo se vio el problema: al traductor se le manda el término suelto, así que su respuesta no pertenece a ninguna acepción concreta. Guardarla en la acepción equivocada la deja mal permanentemente.

La lección. Construir algo es la única forma fiable de auditar su diseño. Y cuando la construcción desmiente al documento, la salida correcta no es callarlo: es anotar la enmienda con su fecha y su motivo.

Commit: 5a0c86c · «docs: enmendar la especificación del diccionario»

7. El mensaje de error del navegador, tal cual

El síntoma. Cuando algo fallaba, la interfaz mostraba el texto crudo de la excepción, escrito en inglés y pensado para un programador.

La causa. El camino de menos resistencia: capturar el error y pintarlo. Nadie decidió eso; simplemente pasó.

La lección. Los mensajes de error son parte del producto y merecen escribirse, no heredarse. Hay dos commits sobre esto: «Decir qué ha fallado en vez de escupir el mensaje del navegador» y «Limpiar el error al reintentar».

Commit: 6f4a222 · 60b957b

SECCIÓN 14

Lo que cuesta y lo que es gratis

Una regla que atraviesa el proyecto entero: **la opción gratuita gana siempre que exista**, y la inteligencia artificial es un botón opcional, no un peaje. Solo dos operaciones de toda la aplicación llaman a la API de pago.

Operación	¿Llama a Claude?	Coste
Extraer vocabulario de un PDF	Sí, obligatoriamente	~0,003 € por término (medido)
«Afinar con IA» en el diccionario	Sí, solo si se pulsa	unos céntimos por palabra
Buscar en el diccionario (escalones 1 a 3)	No, nunca	0 €
Cargar la cola del repaso	No	0 €
Responder una tarjeta	No	0 €
Cambiar los ajustes	No	0 €
Editar la biblioteca	No	0 €
Alojamiento (Vercel) y base de datos (Neon)	—	0 € en plan gratuito

La medición real de la primera extracción, el 6 de septiembre de 2026: **26 términos por unos 0,07 €**. Mil términos en la biblioteca costarían menos de 3 €. Es una cifra que conviene tener presente porque desmonta la intuición de que «usar un modelo grande» es caro: lo caro sería usarlo en el repaso, que se abre veinte veces al día, y por eso el repaso no lo usa.

EL DETALLE MÁS FINO DEL COSTE

El botón de «Afinar con IA» **desaparece en cuanto la acepción está guardada** en la biblioteca. El motivo: afinar reescribe la caché del diccionario, no la ficha ya creada, así que pulsarlo entonces cobraría por un cambio que la tarjeta de repaso no llegaría a ver. Cobrar por algo que no se nota es peor que cobrar caro.

LO QUE APRENDÍ

En un producto que llama a una API de pago, la arquitectura de costes es una decisión de diseño tan importante como el modelo de datos. Aquí se materializó en una regla verificable —las rutas que cuestan están separadas de las que no— y en una escalera de cuatro fuentes que va de lo gratis e instantáneo a lo lento y caro, parándose en la primera que responde.

El presupuesto real del proyecto

0 € alojamiento	0 € base de datos	0 € diccionario	0 € traductor	< 3 € por 1.000 términos
--------------------	----------------------	--------------------	------------------	-----------------------------

SECCIÓN 15

Estado actual, límites conocidos y qué viene después

Qué está terminado

- **Fase 1 — extracción y biblioteca.** Código completo y probado, y verificado contra un PDF real en el despliegue.
- **Fase 2 — repaso con repetición espaciada.** Construida: pantalla de repaso con su pantalla previa, algoritmo FSRS, los tres ajustes y el manifiesto para instalar la app en el móvil.
- **Fase 3 — diccionario.** Fundido en la rama principal y cargado en la base real: 181.103 entradas de Wikcionario.
- **Fase 4 — colecciones y tamaño de sesión.** Fundida, con su migración aplicada.
- **Despliegue.** En Vercel, con despliegue automático al empujar la rama principal.

Los límites que se conocen y están escritos

Límite	Por qué existe	Qué implica
No hay pruebas de componentes	Las pruebas corren en Node, sin navegador simulado	Ningún test puede pulsar un botón. Lo que se prueba de la interfaz son sus funciones puras. Dos fallos reales solo aparecieron a mano.
Contraseña sin protección contra fuerza bruta	Comparación de cadenas normal y sin límite de intentos, a cambio de no añadir estado	Obliga a que la contraseña sea larga y aleatoria, no adivinable.
Si falta el secreto de sesión, el arranque falla con un error genérico	No se añadió una comprobación con mensaje propio	Está avisado en el README: si algo revienta de forma rara al arrancar, comprobar esa variable primero.
La prueba de aceptación del repaso está pendiente	Requiere usar la app en días distintos, no se puede comprobar de un tirón	Queda por confirmar en el móvil que lo valorado «Otra vez» vuelve al día siguiente y lo «Fácil» no.

La prueba de aceptación que solo puede hacer una persona

Merece la pena señalarla porque es una idea de método: hay comprobaciones que ninguna suite puede hacer. La lista de la fase 2 incluye «completar una sesión entera con una mano, sin esperas entre tarjetas» y «que el diseño resulte cómodo tras varias sesiones, no solo bonito la primera vez». Están escritas como casillas en el README, igual que un test, y hasta que no se marquen la fase no se considera terminada.

LO QUE APRENDÍ

Escribir la prueba de aceptación manual con el mismo rigor que una automática —casillas, criterios concretos, y la condición de que la fase no está hecha hasta marcarlas— es lo que impide que «funciona en mi máquina» se convierta en «está terminado».

Qué vendría después

- Cerrar la prueba de aceptación del repaso en el móvil, a lo largo de varios días.

- Pruebas de componentes con un navegador simulado, que es el hueco de cobertura conocido.
- Un mensaje de arranque claro cuando falta el secreto de sesión.

SECCIÓN 16

Glosario

Términos que aparecen en este documento y que conviene tener a mano.

Término	Qué significa aquí
Acepción	Uno de los significados de una palabra. <i>Bank</i> tiene siete en Wikcionario.
App Router	La forma de organizar rutas de Next.js por carpetas, donde conviven página y API.
Atomicidad	Que un conjunto de operaciones ocurra entero o no ocurra.
Base64	Forma de representar datos binarios como texto, para poder enviarlos en un JSON.
Deduplicación	Detectar que dos entradas son la misma y fusionarlas en lugar de duplicarlas.
Driver	La librería que habla con la base de datos por debajo del ORM.
FSRS	<i>Free Spaced Repetition Scheduler</i> : el algoritmo que decide cuándo repasar cada tarjeta.
Idempotente	Que repetir la operación no cambia el resultado.
JSONL	Un fichero con un objeto JSON por línea; se procesa línea a línea.
Lema	La forma de diccionario de una palabra, de la que salen sus variantes.
MCER	Marco Común Europeo de Referencia: los niveles A1 a C2.
Middleware	Código que se ejecuta antes de cada petición; aquí, el que protege las rutas.
Migración	Un cambio en la estructura de la base de datos, guardado como fichero.
NFC	Una de las formas normalizadas de Unicode; unifica caracteres compuestos.
ORM	Capa que traduce entre las tablas de la base y los objetos del lenguaje.
PWA	Aplicación web que se puede instalar en el móvil como si fuera nativa.
Repetición espaciada	Método de estudio que enseña cada cosa justo antes de olvidarla.
Salida estructurada	Pedirle al modelo que rellene un esquema en vez de escribir texto libre.
Token	La unidad en que se mide y se cobra el texto que entra y sale de un modelo.
Transacción	Un bloque de operaciones de base de datos que se confirma o se deshace entero.
Verbo frasal	Verbo + partícula con significado propio: <i>come across</i> , <i>give up</i> .
Worktree	Copia de trabajo adicional del mismo repositorio de git, en otra rama.

APÉNDICE A

Mapa de ficheros

Todo cuelga de la raíz del proyecto; el alias @/* apunta a la raíz.

Lógica pura y utilidades — lib/

Fichero	Responsabilidad
normalize.ts	La clave de comparación de términos
cost.ts	Convertir consumo de tokens en dólares
extraction-schema.ts	El esquema zod del resultado y los tipos compartidos
prompt.ts	Construir el prompt de extracción a partir del nivel
anthropic-extract.ts	Llamar a Claude con el PDF y devolver términos y coste
pdf-slice.ts	Recortar páginas y planificar lotes (navegador)
run-extraction.ts	Orquestar los lotes en secuencia (navegador)
auth.ts	Firmar y verificar la cookie de sesión
fsrs.ts	El puente entre las filas de la base y las tarjetas de ts-fsrs
dia.ts	Dónde empieza «hoy»: la medianoche de Madrid
barajar.ts	El sorteo Fisher-Yates con azar inyectable
review-session.ts	Las funciones puras de la sesión de repaso
repaso/coleccion.ts	Qué entra en la sesión según colección y tamaño
diccionario/lineas.ts	Convertir una línea del volcado en filas
diccionario/lema.ts	Variantes del lema: one/you, someone/somebody
diccionario/traductor.ts	El cliente de MyMemory, con su plazo
diccionario/afinar.ts	La llamada opcional a Claude

Datos — db/

Fichero	Responsabilidad
schema.ts	Las siete tablas, con sus índices y comentarios
client.ts	La conexión a Postgres en producción
types.ts	El tipo Database que aceptan los repositorios
repository/extraction.ts	Guardar una extracción con deduplicación, en transacción
repository/terms.ts	Listar, editar y borrar términos
repository/review.ts	La cola del día y la aplicación de respuestas
repository/settings.ts	Los tres ajustes
repository/diccionario.ts	Búsqueda, caché de traducción y alta desde el diccionario

Pantallas y rutas — app/ y components/

Ruta o componente	Papel
/extraer	Subir el PDF, elegir páginas y nivel, ver el progreso y el coste
/biblioteca	La tabla editable de vocabulario
/repaso	La pantalla previa y la sesión de tarjetas
/diccionario	Buscar una palabra y añadirla
/login	La entrada con contraseña única
api/extract	Extracción de un lote (la única ruta cara y obligatoria)
api/terms, api/terms/[id]	Listar, crear, editar y borrar términos
api/repaso/cola, respuesta, resumen	El repaso: leer la cola, responder e informar. No tocan la API de Claude
api/diccionario	Los escalones 1 a 3 de la búsqueda. Nunca llama a Claude
api/diccionario/afinar	El escalón 4, aparte y a propósito
api/ajustes	Leer y guardar los tres ajustes
proxy.ts	La protección de todas las rutas salvo la de entrada

APÉNDICE B

Comandos útiles

En local

```
npm install
cp .env.example .env.local      # y rellenar las cuatro variables
DATABASE_URL='...' npx drizzle-kit push  # crea las tablas
npm run dev                     # http://localhost:3000
```

Nota importante: drizzle-kit no lee .env.local automáticamente, porque el proyecto no usa dotenv. Hay que pasarle la conexión en el propio comando, tanto en local como contra producción.

Verificar antes de dar algo por terminado

```
npm test          # 440 pruebas, ~43 s
npx tsc --noEmit  # comprobación de tipos
npx eslint .      # estilo y errores comunes
```

Cargar el diccionario (una sola vez)

```
DATABASE_URL='...' npm run cargar:diccionario -- ~/Vocably-diccionario/dicc_todo.jsonl.gz
```

Las cuatro variables de entorno

Variable	Para qué sirve
ANTHROPIC_API_KEY	Autentica las llamadas a la API de Claude
DATABASE_URL	Cadena de conexión de Postgres en Neon
APP_PASSWORD	La única contraseña con la que se entra
SESSION_SECRET	Firma la cookie de sesión. Sin valor por defecto a propósito: si falta, la app debe fallar de forma ruidosa

Ninguna es pública: no hay una sola variable con prefijo NEXT_PUBLIC_ en todo el proyecto. Y .env.local está en .gitignore: nunca debe llegar a un commit.

Documento generado el 9 de septiembre de 2026 a partir del repositorio de Vocably: historial de git, especificaciones, planes de implementación, esquema, migraciones y salida de la suite de pruebas.